

Reviewer Pack — Coarse-Equivalence Invariance of Protocol-Pullback Multiplic...

Entry aa9fe4edbf5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 00:24:11 UTC

Coarse-Equivalence Invariance of Protocol-Pullback Multiplicity Across Bit-Permuted Gadgets

Entry ID: aa9fe4edbf5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 00:24:11 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) — Coarse Geometry / Roe Algebras (within Geometric Group Theory) **Field B** (complexity object): Deterministic 2-party communication complexity, specifically the multiplicity m_Π of the RoeCover extracted from a deterministic protocol's ProtocolPullback for the lifted function $f \circ G^n$

Statement:

Let $G_1 = (X, Y, g, d)$ be a MetricGadget on small alphabet ($|X|, |Y| \leq 4$) and let G_2 be obtained from G_1 by a bi-Lipschitz bijection $\varphi: X \times Y \rightarrow X \times Y$ with distortion ≤ 2 (a coarse equivalence: φ permutes inputs while distorting d by at most a factor 2). Fix any boolean $f: \{0,1\}^n \rightarrow \{0,1\}$ with $n \leq 4$. Then for every deterministic protocol Π_1 computing $f \circ G_1^n$, there exists a deterministic protocol Π_2 computing $f \circ G_2^n$ with cost $\text{cost}(\Pi_2) \leq \text{cost}(\Pi_1) + O(n)$, and the multiplicities of their CoarsePullback covers at scale $R = \text{diam}(G)$ satisfy $m_{\{\Pi_2\}} \leq 2 \cdot m_{\{\Pi_1\}} + 1$. Equivalently, $\text{CLC}(f, G_1)$ and $\text{CLC}(f, G_2)$ agree up to a constant factor independent of f , instantiating Axiom A1 (Coarse-Lift Functoriality) at the level of the multiplicity invariant that A2 connects to communication complexity.

Rationale (proposer's reasoning):

This conjecture directly tests Axiom A1 (Coarse-Lift Functoriality) by asserting that the *multiplicity invariant* m_Π exposed by Axiom A2 (Protocol $\rightarrow$ Cover) is preserved under coarse equivalences of the underlying gadget. If A1 holds, a bi-Lipschitz φ on $X \times Y$ lifts coordinatewise to a bi-Lipschitz map on $(X \times Y)^n$ with the $d_\oplus$ metric (distortion preserved since $\square^1$ products of bi-Lipschitz maps remain bi-Lipschitz), so any Roe-style cover at scale R for the G_1 -lift can be transported to a cover at scale $\leq 2R$ for the G_2 -lift while at most doubling the multiplicity (each transported set may merge with one neighbor at the boundary, contributing the $+1$ slack). The $O(n)$ cost overhead in Π_2 reflects simulating φ coordinatewise. Verifying this empirically would falsify A1 if multiplicities diverge, and would strengthen confidence that CLC is genuinely a coarse invariant — a prerequisite for using it as a lower-bound technique against communication complexity.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3d376593f7269fdd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all tested (f, Π_1, φ) triples for $n \in \{2, 3, 4\}$, $|X|=|Y|=2$, 16 sampled boolean functions per n , and all distortion- ≤ 2 bijections φ on $X \times Y$, the multiplicity ratio $m_{\{\Pi_2\}}/m_{\{\Pi_1\}}$ must satisfy $m_{\{\Pi_2\}} \leq 2 \cdot m_{\{\Pi_1\}} + 1$ in $\geq 99\%$ of triples, with empirical max ratio ≤ 2.5 aggregated over 5 random seeds for the function/protocol sample.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - coarse equivalence Roe algebra communication complexity gadget composition - lifted function gadget deterministic protocol coarse geometry multiplicity invariant - bi-Lipschitz gadget composition lower bounds protocol partition Roe cover

Top relevant hits considered: - [<http://arxiv.org/abs/2408.07701v2>] A categorical interpretation of Morita equivalence for dynamical von Neumann algebras - [<http://arxiv.org/abs/2312.08907v2>] Roe algebras of coarse spaces via coarse geometric modules - [<http://arxiv.org/abs/1706.00276v1>] Counting coarse subsets of a countable group - [<http://arxiv.org/abs/2110.02008v2>] Lifted Reed-Solomon Codes and Lifted Multiplicity Codes - [<http://arxiv.org/abs/1903.06084v8>] Lifting coarse homotopies - [<http://arxiv.org/abs/1905.02270v4>] Lifted multiplicity codes and the disjoint repair group property - [<http://arxiv.org/abs/1206.5941v1>] Kernelization Lower Bounds By Cross-Composition - [<http://arxiv.org/abs/cond-mat/9601083v2>] Upper and Lower Bounds on the Partition Function of the Hofstadter Model

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from itertools import product, combinations

def dist(a, b):
    return max(abs(ai - bi) for ai, bi in zip(a, b))

def is_bi_lipschitz(phi, distortion=2):
    X = set(x for x, y in phi.keys())
```

```

Y = set(y for x, y in phi.keys())
for (x1, y1), (x2, y2) in product(product(X, Y), repeat=2):
    if dist(phi[(x1, y1)], phi[(x2, y2)]) > distortion * dist((x1, y1), (x2, y2)):
        return False
return True

def generate_bi_lipschitz_permutations():
    X = {0, 1}
    Y = {0, 1}
    permutations = list(product(X, repeat=4))
    valid_perms = []
    for perm in permutations:
        phi = {(x, y): (perm[2*x + y]) for x, y in product(X, Y)}
        if is_bi_lipschitz(phi):
            valid_perms.append(phi)
    return valid_perms

def generate_boolean_functions(n):
    return list(product([0, 1], repeat=n))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    X = {0, 1}
    Y = {0, 1}
    G_1 = (X, Y, lambda x, y: x * y % 2, lambda x, y: abs(x - y))

    phi_perms = generate_bi_lipschitz_permutations()
    n_values = [2, 3, 4]
    boolean_functions = generate_boolean_functions(4)

    metric_value = 0
    instances_tested = 0

    for n in n_values:
        for f in boolean_functions:
            G_1_n = (X, Y, lambda x, y: sum(f[i] * g(x[i], y[i]) for i in range(n)) % 2, lambda x,
            R = 2
            diam_G_1_n = R

            protocols = []
            for _ in range(30): # Sample 30 protocols per (n, f)
                protocol = {}
                for x in product(X, repeat=n):
                    protocol[x] = random.choice([0, 1])
                protocols.append(protocol)

            for protocol in protocols:
                transcript_partition = {}
                for x in product(X, repeat=n):
                    transcript_partition.setdefault(tuple(sorted(protocol[x])), []).append(x)

                m_Pi_1 = max(len(list(group)) for _, group in transcript_partition.items())
                metric_value += m_Pi_1
                instances_tested += 1

```

```

Pi_2 = {}
for x in product(X, repeat=n):
    Pi_2[x] = protocol[phi_perms[random.randint(0, len(phi_perms) - 1)][x]]

transcript_partition_Pi_2 = {}
for x in product(X, repeat=n):
    transcript_partition_Pi_2.setdefault(tuple(sorted(Pi_2[x])), []).append(x)

m_Pi_2 = max(len(list(group)) for _, group in transcript_partition_Pi_2.items())
if m_Pi_2 > 2 * m_Pi_1 + 1:
    return {
        "metric_name": "Multiplicity Ratio",
        "metric_value": m_Pi_2 / m_Pi_1,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": f"m_{Pi_2} > 2 * m_{Pi_1} + 1"
    }

mean_value = metric_value / instances_tested
support_fraction = instances_tested / (len(n_values) * len(boolean_functions) * 30)

return {
    "metric_name": "Multiplicity Ratio",
    "metric_value": mean_value,
    "instances_tested": instances_tested,
    "conjecture_holds": True if support_fraction >= 0.99 else False,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 53)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Multiplicity Ratio > 2 * m_{Pi_1} + 1\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a94fc38.py", line 113, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a94fc38.py", line 49, in run_trial
    phi_perms = generate_bi_lipschitz_permutations()
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a94fc38.py", line 35, in generate_bi_l
    if is_bi_lipschitz(phi):
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a94fc38.py", line 24, in is_bi_lipschi
    if dist(phi[(x1, y1)], phi[(x2, y2)]) > distortion * dist((x1, y1), (x2, y2)):
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8a94fc38.py", line 18, in dist
    return max(abs(ai - bi) for ai, bi in zip(a, b))
            ~~~~~
```

TypeError: 'int' object is not iterable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a TypeError in the bi-Lipschitz check (dist() called on integers instead of tuples) before any trials produced data, so neither the support nor falsification criteria can be evaluated. | next: Fix the dist() helper to handle the (x,y) pair representation correctly (ensure phi maps tuples to tuples and dist unpacks both arguments), then rerun the 5-seed sweep.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	6804
2	novelty	claude_max	opus	0	0	4981
3	novelty	claude_max	opus	0	0	10591
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15650
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21172
6	judge	claude_max	opus	0	0	5700

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 64899 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/aa9fe4edbfe5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aa9fe4edbf5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/aa9fe4edbf5.tar.gz` (if generated)